

HTML DOM (Document Object Model)

1. Pengertian HTML DOM

HTML DOM (Document Object Model) adalah representasi struktur halaman web dalam bentuk objek yang dibuat oleh browser setelah file HTML diproses. Melalui DOM, setiap elemen pada halaman web seperti tombol, input, tabel, gambar, dan teks dapat diakses serta dimanipulasi menggunakan JavaScript.

DOM berfungsi sebagai penghubung antara halaman web dan JavaScript sehingga website dapat memberikan respon terhadap setiap tindakan pengguna, seperti menekan tombol Login, mengisi form, atau memilih menu.

Dalam dunia Quality Assurance (QA), pemahaman mengenai HTML DOM sangat penting karena hampir seluruh aktivitas pengujian web berhubungan langsung dengan elemen yang terdapat pada DOM. Baik saat melakukan Manual Testing menggunakan Chrome DevTools maupun Automation Testing menggunakan Playwright, Selenium, atau Cypress, QA akan selalu berinteraksi dengan struktur DOM.

HTML DOM dalam Pengujian QA

Seorang QA tidak hanya bertugas menemukan bug, tetapi juga memahami bagaimana aplikasi bekerja. Salah satu cara untuk memahami aplikasi web adalah dengan mengenali struktur HTML DOM. Dalam Manual Testing, QA sering menggunakan Chrome DevTools untuk melakukan inspeksi terhadap elemen HTML. Dengan DevTools, QA dapat melihat atribut seperti id, class, name, type, maupun disabled sehingga lebih mudah mengetahui penyebab suatu bug.

Pada Automation Testing, HTML DOM digunakan untuk menemukan locator yang nantinya dipakai oleh Playwright, Selenium, atau Cypress agar script dapat berinteraksi dengan elemen yang benar.

Sebagai contoh, ketika tombol Login tidak dapat diklik, QA dapat memeriksa apakah tombol tersebut memiliki atribut disabled, apakah elemen masih berada di dalam DOM, atau apakah terdapat elemen lain yang menutupi tombol tersebut.

Aktivitas QA yang menggunakan HTML DOM

- Melakukan inspeksi elemen menggunakan Chrome DevTools.
- Memastikan atribut elemen sesuai dengan kebutuhan aplikasi.
- Menentukan locator untuk Automation Testing.
- Menganalisis bug pada tampilan (UI Bug).
- Memastikan perubahan elemen setelah pengguna melakukan suatu aksi.

Validasi Menggunakan Chrome DevTools

Selain melakukan pengujian melalui tampilan aplikasi, QA juga dapat memvalidasi perubahan menggunakan Chrome DevTools.

Langkah-langkah:

1. Buka halaman Login.
2. Tekan tombol F12.
3. Pilih tab Elements.
4. Klik ikon Inspect.
5. Pilih tombol Login.
6. Periksa atribut berikut:
 - id
 - class
 - disabled
 - hidden
7. Pastikan atribut berubah sesuai kondisi aplikasi setelah login berhasil

Hubungan HTML DOM dengan QA

HTML DOM memiliki peran yang sangat penting dalam pekerjaan seorang Quality Assurance.

Manual Testing

- Membantu melakukan inspeksi elemen menggunakan Chrome DevTools.
- Memastikan tampilan aplikasi sesuai dengan desain.
- Memverifikasi atribut HTML pada setiap elemen.
- Menganalisis penyebab bug pada User Interface.

Automation Testing

- Menentukan locator menggunakan ID, Class, atau CSS Selector.
- Membantu Playwright dan Selenium menemukan elemen yang akan diuji.
- Mengurangi risiko kegagalan automation akibat perubahan struktur halaman.

Bug Analysis

Dengan memahami HTML DOM, QA dapat mengetahui apakah bug disebabkan oleh:

- Perubahan atribut elemen.
- Kesalahan JavaScript.
- Perubahan struktur halaman.
- Elemen yang tidak berhasil dimuat.

Pemahaman terhadap DOM juga mempermudah komunikasi antara QA dan Developer karena QA dapat memberikan informasi yang lebih spesifik mengenai letak permasalahan.

Ringkasan

HTML DOM merupakan representasi struktur halaman web yang memungkinkan browser dan JavaScript berinteraksi dengan elemen HTML. Bagi seorang QA, memahami HTML DOM sangat penting karena digunakan dalam inspeksi elemen, analisis bug, validasi tampilan aplikasi, serta pembuatan automation testing menggunakan Playwright, Selenium, maupun Cypress.

Dengan memahami konsep HTML DOM, seorang QA dapat melakukan pengujian secara lebih efektif, menemukan penyebab bug dengan lebih cepat, dan berkomunikasi dengan Developer menggunakan istilah teknis yang tepat.

2. Pengertian CSS Selector

CSS Selector adalah pola atau cara untuk memilih elemen HTML berdasarkan atribut tertentu, seperti id, class, tag, maupun attribute. Pada awalnya CSS Selector digunakan untuk memberikan tampilan (style) pada halaman web. Namun, dalam dunia Quality Assurance, CSS Selector memiliki peran yang jauh lebih penting, yaitu sebagai locator untuk menemukan elemen pada proses automation testing.

Framework automation seperti Playwright, Selenium, dan Cypress menggunakan CSS Selector agar script dapat menemukan elemen yang akan diuji. Dengan locator yang tepat, automation testing dapat berjalan lebih stabil dan akurat.

Selain untuk automation testing, CSS Selector juga membantu QA saat melakukan inspeksi elemen menggunakan Chrome DevTools untuk memastikan bahwa elemen yang diuji sesuai dengan yang ditampilkan di halaman web.

CSS Selector dalam Pengujian QA

Dalam pekerjaan sehari-hari, seorang QA akan sering menggunakan CSS Selector untuk mencari elemen pada halaman web.

Sebagai contoh, ketika QA ingin melakukan automation testing pada halaman Login, script harus mengetahui letak textbox Email, textbox Password, dan tombol Login. CSS Selector menjadi salah satu cara tercepat untuk menemukan elemen tersebut.

Selain itu, CSS Selector juga digunakan saat melakukan debugging menggunakan Chrome DevTools. Dengan menekan Ctrl + F pada tab Elements, QA dapat mencari elemen menggunakan CSS Selector untuk memastikan elemen tersebut benar-benar ada di dalam DOM.

Aktivitas QA yang menggunakan CSS Selector

- Menentukan locator pada Playwright atau Selenium.
- Melakukan inspeksi elemen menggunakan Chrome DevTools.
- Memastikan locator tetap valid setelah aplikasi diperbarui.
- Menganalisis penyebab automation test gagal akibat perubahan elemen.

Test Scenario

Memastikan CSS Selector yang digunakan pada automation masih sesuai dengan struktur halaman.

Test Steps

1. Buka halaman Login.
2. Tekan F12.
3. Pilih tab Elements.
4. Tekan Ctrl + F.
5. Ketik selector berikut: .btn-login
6. Pastikan browser berhasil menemukan tombol Login.

Expected Result

- Elemen Login berhasil ditemukan.
- CSS Selector masih valid.
- Automation dapat dijalankan tanpa error.

Hubungan CSS Selector dengan QA

CSS Selector memiliki peran yang sangat penting dalam pekerjaan seorang QA, khususnya pada Automation Testing.

Manual Testing

- Membantu mencari elemen menggunakan Chrome DevTools.
- Memastikan atribut elemen tidak berubah setelah aplikasi diperbarui.
- Memvalidasi struktur HTML sebelum automation dibuat.

Automation Testing

- Digunakan sebagai locator pada Playwright, Selenium, dan Cypress.
- Membantu script menemukan elemen secara cepat dan akurat.
- Mengurangi penggunaan XPath yang cenderung lebih kompleks.

Bug Analysis

QA dapat mengetahui penyebab automation gagal apabila:

- ID elemen berubah.
- Class elemen berubah.
- Elemen dihapus dari halaman.
- Struktur HTML berubah setelah update aplikasi.

Dengan memahami CSS Selector, QA dapat memperbaiki locator lebih cepat dan mengurangi waktu debugging.

Ringkasan

CSS Selector merupakan cara untuk memilih elemen HTML berdasarkan atribut tertentu. Dalam dunia QA, CSS Selector digunakan sebagai locator pada automation testing serta membantu proses inspeksi elemen menggunakan Chrome DevTools. Pemahaman terhadap CSS Selector akan membantu QA membuat automation testing yang lebih stabil, mempercepat proses debugging, serta mempermudah komunikasi dengan Developer ketika terjadi perubahan pada struktur halaman web.

3. Pengertian AJAX

AJAX (Asynchronous JavaScript and XML) adalah teknik pada aplikasi web yang memungkinkan browser mengirim dan menerima data dari server secara asynchronous, sehingga halaman web tidak perlu dimuat ulang (reload) setiap kali terjadi pertukaran data. Pada website modern, hampir semua aktivitas menggunakan AJAX, seperti login, pencarian produk, filter data, notifikasi, hingga proses checkout. Ketika pengguna melakukan suatu aksi, browser hanya mengirim data yang diperlukan ke server dan menerima respons, kemudian memperbarui bagian tertentu dari halaman tanpa me-refresh seluruh halaman. Meskipun

nama AJAX mengandung kata XML, saat ini sebagian besar aplikasi web menggunakan format JSON sebagai media pertukaran data karena lebih ringan dan mudah diproses. Bagi seorang Quality Assurance (QA), memahami konsep AJAX sangat penting karena banyak bug yang berkaitan dengan proses komunikasi antara browser dan server.

AJAX dalam Pengujian QA

Dalam pengujian aplikasi web, QA tidak hanya memeriksa tampilan antarmuka (UI), tetapi juga perlu memastikan bahwa data berhasil dikirim ke server dan respons yang diterima sesuai dengan yang diharapkan.

Untuk melakukan hal tersebut, QA biasanya menggunakan Chrome DevTools pada tab Network. Melalui tab ini, QA dapat melihat request yang dikirim, response yang diterima, status code, waktu respons, serta payload yang dikirimkan ke server.

Sebagai contoh, ketika pengguna melakukan login, QA dapat memastikan bahwa browser mengirimkan email dan password ke endpoint login, kemudian memeriksa apakah server memberikan respons sukses atau gagal.

Aktivitas QA yang menggunakan AJAX

- Memeriksa request yang dikirim ke server.
- Memastikan response sesuai dengan data yang diharapkan.
- Memvalidasi status code dari server.
- Menganalisis bug ketika data tidak muncul pada halaman.
- Mengidentifikasi error API melalui Chrome DevTools.

Test Scenario

Memastikan request login berhasil dikirim dan response diterima dengan benar.

Test Steps

1. Buka halaman Login.
2. Tekan F12 untuk membuka Chrome DevTools.
3. Pilih tab Network.
4. Aktifkan filter Fetch/XHR.
5. Masukkan email dan password yang valid.
6. Klik tombol Login.
7. Pilih request login yang muncul pada daftar Network.
8. Periksa Request Payload, Response, dan Status Code.

Expected Result

- Request berhasil dikirim ke endpoint login.
- Payload berisi email dan password.
- Response berisi data pengguna atau token autentikasi.
- Status Code 200 OK.
- Pengguna berhasil masuk ke Dashboard

Hubungan AJAX dengan QA

AJAX merupakan salah satu konsep yang sangat penting bagi seorang Quality Assurance karena berhubungan langsung dengan proses pertukaran data antara browser dan server.

Manual Testing

- Memeriksa request dan response menggunakan Chrome DevTools.
- Memastikan data berhasil dikirim ke server.
- Memastikan data yang diterima sesuai dengan yang ditampilkan pada halaman.

Automation Testing

- Memvalidasi response API menggunakan Playwright.
- Menunggu request selesai sebelum melanjutkan proses pengujian.
- Memastikan data yang diterima sesuai dengan skenario pengujian.

Bug Analysis

Dengan memahami AJAX, QA dapat mengetahui apakah bug disebabkan oleh:

- Request tidak terkirim.
- Response dari server tidak sesuai.
- Status Code error (400, 401, 404, 500).
- JavaScript gagal memproses response.
- API membutuhkan waktu terlalu lama untuk merespons.

Pemahaman terhadap AJAX juga membantu QA membedakan apakah masalah berasal dari **Frontend**, **Backend**, atau **Network**, sehingga proses pelaporan bug menjadi lebih akurat.

Ringkasan

AJAX adalah teknik yang memungkinkan browser mengirim dan menerima data dari server tanpa me-refresh halaman. Dalam dunia QA, pemahaman terhadap AJAX sangat penting karena membantu memverifikasi proses komunikasi antara browser dan server. Dengan memahami AJAX, seorang QA dapat menganalisis request, response, status code, serta menentukan apakah suatu bug berasal dari frontend, backend, atau jaringan. Pengetahuan ini akan meningkatkan kualitas pengujian baik pada Manual Testing maupun Automation Testing menggunakan Playwright, Selenium, atau alat pengujian lainnya.

4. Pengertian Cache

Cache adalah mekanisme penyimpanan data sementara yang digunakan oleh browser maupun server untuk mempercepat proses pemuatan (loading) halaman web. Ketika pengguna pertama kali membuka sebuah website, browser akan mengunduh berbagai file seperti HTML, CSS, JavaScript, gambar, dan font. File-file tersebut kemudian disimpan pada cache sehingga ketika pengguna membuka halaman yang sama di kemudian hari, browser tidak perlu mengunduh ulang seluruh file dari server.

Dengan adanya cache, waktu loading menjadi lebih cepat dan penggunaan bandwidth menjadi lebih efisien. Namun, bagi seorang Quality Assurance (QA), cache juga dapat menjadi penyebab munculnya hasil pengujian yang tidak sesuai karena browser masih menampilkan file lama meskipun developer telah melakukan perubahan pada aplikasi.

Oleh karena itu, seorang QA perlu memahami cara kerja cache agar dapat memastikan bahwa aplikasi yang diuji menggunakan versi terbaru.

Cache dalam Pengujian QA

Dalam proses pengujian aplikasi web, cache sering kali menjadi penyebab munculnya bug semu (false bug). Sebagai contoh, developer telah memperbaiki tampilan tombol Login dan melakukan deployment ke server, tetapi saat QA membuka aplikasi, perubahan tersebut belum terlihat karena browser masih menggunakan file JavaScript atau CSS yang tersimpan di cache. Untuk menghindari kondisi tersebut, QA biasanya melakukan Hard Reload, Empty Cache and Hard Reload, atau mengaktifkan opsi Disable Cache pada Chrome DevTools selama proses pengujian.

Selain itu, QA juga perlu memastikan apakah bug yang ditemukan benar-benar berasal dari aplikasi atau hanya disebabkan oleh data cache yang belum diperbarui.

Aktivitas QA yang menggunakan Cache

- Melakukan Hard Reload setelah deployment aplikasi.
- Menghapus cache browser sebelum regression testing.
- Mengaktifkan opsi Disable Cache saat debugging.

- Memastikan perubahan terbaru dari developer sudah tampil pada aplikasi.
- Menghindari laporan bug yang disebabkan oleh cache lama.

Hubungan Cache dengan QA

Pemahaman mengenai cache sangat penting dalam pekerjaan QA karena berkaitan langsung dengan validitas hasil pengujian.

Manual Testing

- Menghindari hasil pengujian yang menggunakan file lama.
- Memastikan perubahan dari developer sudah tampil.
- Melakukan Hard Reload sebelum menguji fitur baru.

Automation Testing

- Memastikan automation berjalan menggunakan versi aplikasi terbaru.
- Menghindari kegagalan automation akibat file lama yang tersimpan di browser.

Bug Analysis

Dengan memahami cache, QA dapat membedakan apakah masalah yang ditemukan benar-benar berasal dari aplikasi atau hanya disebabkan oleh browser yang masih menggunakan data lama.

Hal ini membantu mengurangi laporan bug yang tidak valid dan mempercepat proses verifikasi hasil perbaikan.

5. Pengertian Browser Add-ons

Browser Add-ons atau Browser Extensions merupakan perangkat lunak tambahan yang dapat dipasang pada browser untuk menambahkan fitur atau fungsi tertentu sesuai dengan kebutuhan pengguna. Browser Add-ons dapat digunakan untuk meningkatkan produktivitas, membantu proses debugging, melakukan analisis aplikasi web, serta mempermudah proses pengujian yang dilakukan oleh seorang Quality Assurance (QA).

Dalam proses pengujian aplikasi web, Browser Add-ons berperan sebagai alat bantu untuk memperoleh informasi yang tidak dapat dilihat secara langsung dari antarmuka aplikasi. Misalnya, QA dapat menggunakan Browser Add-ons untuk melihat struktur data JSON, memodifikasi HTTP Header, mengidentifikasi teknologi yang digunakan oleh aplikasi, hingga melakukan analisis performa website.

Oleh karena itu, pemanfaatan Browser Add-ons menjadi salah satu keterampilan yang perlu dimiliki oleh QA, baik pada pengujian manual maupun automation testing.

Browser Add-ons dalam Pengujian QA

Dalam aktivitas pengujian, QA sering kali membutuhkan informasi tambahan yang tidak tersedia secara langsung pada browser. Browser Add-ons dapat membantu QA memperoleh informasi tersebut sehingga proses analisis dan debugging menjadi lebih mudah.

Sebagai contoh, ketika melakukan pengujian API, response yang diterima dari server umumnya berbentuk JSON. Tanpa bantuan Browser Add-ons, data JSON akan ditampilkan dalam bentuk teks yang sulit dibaca. Dengan menggunakan ekstensi seperti JSON Formatter, data tersebut dapat ditampilkan dalam format yang lebih terstruktur sehingga lebih mudah dianalisis.

Selain itu, QA juga dapat menggunakan Browser Add-ons untuk mengubah request header, mengidentifikasi framework yang digunakan oleh aplikasi, maupun mengukur performa website.

Aktivitas QA yang menggunakan Browser Add-ons

- Membaca dan menganalisis response API.
- Melakukan modifikasi HTTP Header untuk kebutuhan pengujian.
- Mengidentifikasi teknologi yang digunakan oleh aplikasi.
- Melakukan analisis performa website.
- Membantu proses debugging selama pengujian.

Hubungan Browser Add-ons dengan QA

Browser Add-ons memiliki peran yang penting dalam mendukung proses pengujian aplikasi web.

Manual Testing

- Membantu membaca response API.
- Mempermudah analisis struktur data JSON.
- Membantu identifikasi teknologi yang digunakan aplikasi.
- Mendukung proses analisis performa website.

Automation Testing

- Membantu memahami request dan response sebelum membuat automation script.
- Mempermudah validasi data hasil automation testing.

Bug Analysis

Browser Add-ons membantu QA memperoleh informasi yang lebih lengkap ketika melakukan investigasi terhadap suatu bug, sehingga laporan yang diberikan kepada Developer menjadi lebih akurat dan mudah dipahami.

Beberapa Browser Add-ons yang direkomendasikan untuk digunakan oleh seorang QA antara lain:

- **JSON Formatter**, untuk membaca response API dalam format yang lebih terstruktur.
- **ModHeader**, untuk memodifikasi HTTP Header pada request.
- **Wappalyzer**, untuk mengidentifikasi teknologi yang digunakan oleh aplikasi.
- **Lighthouse**, untuk melakukan analisis performa dan kualitas website.

Pemilihan Browser Add-ons yang sesuai dapat meningkatkan efisiensi proses pengujian dan mempercepat proses identifikasi permasalahan pada aplikasi.

6. Pengertian Client-Side Rendering (CSR) dan Server-Side Rendering (SSR)

Client-Side Rendering (CSR) adalah metode rendering halaman web di mana proses pembuatan tampilan dilakukan oleh browser menggunakan JavaScript setelah halaman diterima dari server. Pada metode ini, server hanya mengirimkan file HTML dasar beserta file JavaScript yang kemudian akan diproses oleh browser untuk membangun tampilan aplikasi.

Sebaliknya, Server-Side Rendering (SSR) merupakan metode rendering di mana seluruh proses pembuatan halaman dilakukan di sisi server. Browser akan menerima halaman HTML yang sudah lengkap sehingga konten dapat langsung ditampilkan tanpa harus menunggu proses rendering dari JavaScript.

Dalam pengembangan aplikasi web modern, kedua metode ini memiliki karakteristik yang berbeda. Oleh karena itu, seorang Quality Assurance (QA) perlu memahami perbedaan antara CSR dan SSR karena metode rendering dapat memengaruhi proses pengujian aplikasi, terutama pada aspek performa, tampilan, serta automation testing.

CSR dan SSR dalam Pengujian QA

Pemahaman mengenai CSR dan SSR membantu QA dalam menentukan strategi pengujian yang tepat. Pada aplikasi yang menggunakan CSR, elemen halaman sering kali membutuhkan waktu beberapa saat hingga muncul karena proses rendering dilakukan setelah browser menjalankan JavaScript. Kondisi ini dapat menyebabkan automation testing gagal apabila script langsung mencari elemen tanpa menunggu proses rendering selesai.

Sebaliknya, pada aplikasi yang menggunakan SSR, halaman umumnya dapat ditampilkan lebih cepat karena seluruh konten telah diproses oleh server sebelum dikirim ke browser.

Dalam proses pengujian, QA perlu memastikan bahwa seluruh elemen berhasil dimuat sebelum melakukan validasi terhadap fitur aplikasi.

Aktivitas QA yang berkaitan dengan CSR dan SSR

- Memastikan seluruh elemen halaman berhasil dimuat.
- Menganalisis waktu loading halaman.
- Mengidentifikasi penyebab elemen tidak ditemukan pada automation testing.
- Memastikan data ditampilkan dengan benar setelah proses rendering selesai.
- Menyesuaikan strategi automation berdasarkan metode rendering aplikasi.

Perbedaan CSR dan SSR

Aspek	Client-Side Rendering (CSR)	Server-Side Rendering (SSR)
Proses Rendering	Dilakukan di browser	Dilakukan di server
Waktu Tampilan Awal	Relatif lebih lambat	Lebih cepat
Penggunaan JavaScript	Sangat bergantung pada JavaScript	Tidak bergantung sepenuhnya pada JavaScript
SEO	Kurang optimal	Lebih optimal
Pengaruh terhadap Automation	Memerlukan proses waiting	Elemen umumnya langsung tersedia

Hubungan CSR dan SSR dengan QA

Pemahaman mengenai CSR dan SSR memberikan manfaat dalam proses pengujian aplikasi web.

Manual Testing

- Membantu memahami penyebab halaman membutuhkan waktu untuk menampilkan data.
- Mempermudah analisis apabila data tidak muncul setelah halaman dibuka.
- Membantu melakukan validasi terhadap proses rendering halaman.

Automation Testing

- Menentukan strategi waiting sebelum melakukan validasi.
- Mengurangi kegagalan automation akibat elemen yang belum selesai dimuat.

- Membantu memilih locator dan waktu eksekusi yang tepat.

Bug Analysis

Dengan memahami metode rendering yang digunakan aplikasi, QA dapat mengidentifikasi apakah suatu permasalahan disebabkan oleh proses rendering, kesalahan JavaScript, atau kegagalan pengambilan data dari server.

Hal tersebut membantu QA dalam memberikan laporan bug yang lebih akurat kepada tim pengembang.

7. Pengertian Responsive Web Design dan Adaptive Web Design

Responsive Web Design (RWD) adalah pendekatan dalam pengembangan website yang memungkinkan tampilan aplikasi menyesuaikan ukuran layar perangkat secara otomatis menggunakan satu layout yang fleksibel. Dengan pendekatan ini, halaman web dapat memberikan pengalaman pengguna yang optimal pada berbagai perangkat, seperti desktop, tablet, maupun smartphone.

Adaptive Web Design (AWD) merupakan pendekatan yang menggunakan beberapa layout berbeda untuk ukuran layar tertentu. Ketika pengguna membuka website, sistem akan memilih layout yang paling sesuai dengan ukuran perangkat yang digunakan.

Kedua pendekatan tersebut bertujuan untuk meningkatkan kenyamanan pengguna dalam mengakses aplikasi. Oleh karena itu, seorang Quality Assurance (QA) perlu memastikan bahwa tampilan aplikasi tetap berfungsi dengan baik pada berbagai ukuran layar.

Responsive dan Adaptive dalam Pengujian QA

Pengujian responsive merupakan salah satu aktivitas penting dalam pengujian aplikasi web. QA harus memastikan bahwa seluruh komponen antarmuka, seperti tombol, menu, gambar, tabel, dan form, tetap dapat digunakan dengan baik pada berbagai resolusi layar.

Selain memeriksa tampilan, QA juga perlu memastikan bahwa fungsi aplikasi tetap berjalan normal meskipun ukuran layar berubah.

Beberapa browser menyediakan fitur **Device Toolbar** pada Chrome DevTools yang memungkinkan QA melakukan simulasi berbagai jenis perangkat tanpa harus menggunakan perangkat fisik.

Aktivitas QA yang berkaitan dengan Responsive Testing

- Menguji tampilan pada desktop, tablet, dan smartphone.
- Memastikan seluruh tombol dapat diakses.
- Memastikan teks mudah dibaca.

- Memastikan gambar tidak terpotong.
- Memastikan form tetap dapat digunakan pada berbagai ukuran layar.

Hubungan Responsive Web Design dengan QA

Responsive testing merupakan bagian dari pengujian antarmuka (UI Testing) yang bertujuan memastikan aplikasi dapat digunakan pada berbagai perangkat.

Manual Testing

- Memastikan tampilan sesuai ukuran layar.
- Memastikan fungsi aplikasi tetap berjalan.
- Memastikan navigasi mudah digunakan.

Automation Testing

- Melakukan pengujian menggunakan berbagai ukuran viewport.
- Memastikan layout tetap sesuai setelah perubahan aplikasi.
- Mendukung regression testing pada berbagai resolusi layar.

Bug Analysis

QA dapat mengidentifikasi bug yang hanya muncul pada perangkat tertentu, seperti tombol yang tidak dapat diklik, teks yang bertumpuk, atau gambar yang tidak tampil sempurna.

8. Pengertian Progressive Web App (PWA)

Progressive Web App (PWA) merupakan aplikasi web yang dirancang agar memiliki pengalaman penggunaan menyerupai aplikasi mobile. PWA memanfaatkan teknologi modern seperti **Service Worker**, **Web App Manifest**, dan **HTTPS** sehingga aplikasi dapat diakses secara lebih cepat, dapat dipasang (install) pada perangkat pengguna, serta mampu berjalan dalam kondisi jaringan yang terbatas.

Berbeda dengan aplikasi web konvensional, PWA memberikan pengalaman penggunaan yang lebih baik karena mendukung fitur seperti notifikasi, akses offline, dan proses loading yang lebih cepat. Bagi seorang Quality Assurance (QA), pemahaman mengenai PWA diperlukan untuk memastikan seluruh fitur tersebut berjalan sesuai dengan kebutuhan pengguna.

Progressive Web App dalam Pengujian QA

Pengujian terhadap PWA tidak hanya berfokus pada tampilan aplikasi, tetapi juga mencakup fitur-fitur tambahan yang membedakan PWA dengan website biasa.

QA perlu memastikan bahwa aplikasi dapat dipasang pada perangkat, dapat diakses kembali melalui ikon aplikasi, serta tetap berfungsi ketika koneksi internet tidak stabil.

Selain itu, QA juga perlu memverifikasi apakah Service Worker bekerja dengan benar dalam menyimpan data yang diperlukan untuk penggunaan offline.

Aktivitas QA yang berkaitan dengan PWA

- Memastikan aplikasi dapat dipasang pada perangkat.
- Menguji fungsi aplikasi dalam kondisi offline.
- Memverifikasi notifikasi (Push Notification).
- Memastikan proses sinkronisasi data berjalan dengan baik.
- Menguji performa aplikasi menggunakan Lighthouse.

Hubungan Progressive Web App dengan QA

Pengujian terhadap PWA bertujuan memastikan seluruh fitur tambahan berjalan sesuai dengan kebutuhan pengguna.

Manual Testing

- Menguji proses instalasi aplikasi.
- Memastikan aplikasi dapat dijalankan melalui ikon pada perangkat.
- Menguji fitur offline.
- Menguji Push Notification.

Automation Testing

- Memastikan fitur utama tetap berjalan setelah aplikasi dipasang.
- Melakukan regression testing terhadap fitur offline.
- Memverifikasi perubahan setelah deployment aplikasi.

Bug Analysis

QA dapat mengidentifikasi permasalahan yang berkaitan dengan Service Worker, cache aplikasi, proses instalasi, maupun sinkronisasi data sehingga pengembang memperoleh informasi yang lebih lengkap dalam proses perbaikan.

